Python Engineering - Class 09

Modules and Imports | EX17 - Vertical Motion Modules

Class objective: Separate responsibilities across Python files and understand imports, main(), __name__ and execution flow.

This document is the base exercise guide. It is not a full solution to copy: it organizes the problem, implementation steps, reference values and required validation.

Path

```
01_python_bootcamp/
`-- week_03_modules_and_coordinates/class_09_modules_imports/
    |-- notes/
    `-- exercices/
```

Required workflow

```
PROBLEM
INPUTS
OUTPUTS
EQUATIONS
SPECIAL CASES
DO I NEED ITERATION?
ALGORITHM
VALIDATION
```

Before coding, write these eight sections. The goal is to separate problem reasoning from syntax.

EX17 - Modular Vertical Motion

Problem: Split the simulation into vertical_motion.py, data_processing.py and main.py.

Reference inputs: same vertical-motion configuration and trajectory

Equations / relationship: Same Euler/processing equations from previous weeks

Expected check: Imported modules should not execute main workflow automatically; process_trajectory returns data; each module imports its own dependencies.

Implementation guide

- 1. Move physics to vertical_motion.py.
- 2. Move processing to data_processing.py.
- 3. Import functions in main.py.
- 4. Define main().
- 5. Protect execution with __name__ guard.

Before running

Predict sign, order of magnitude, data type and any relevant shape. Then compare the prediction with the actual output.

Final validation

Answer these questions as comments at the end of the exercise. The goal is not memorized wording but demonstrating that you can explain the code flow.

- What is a module?
- What happens when Python imports a source file?
- Difference between import module and from module import function.
- What is __name__ when a file is executed directly?
- Why should process_trajectory return data instead of printing it?
- What is the responsibility of main.py?

Completion criterion

The class is complete when the code runs, the reference checks make sense, you can explain every block without reading a solution, and you can modify at least one input while predicting the expected effect.